Clase 14 · SO_REUSEADDR, lectura en bucle y tamaño de búfer

Arquitectura Cliente-Servidor · Mtro. Carlos Alberto Román Zamitiz · martes 13 oct 2026 · Tema 2 · Apuntes y resumen de estudio

PARTE 1 · Apuntes completos

"Practiquen los programas 04, 05 y compara_tiempos_buffers… el examen es muy parecido al cuatro y al cinco."

1. setsockopt y SO_REUSEADDR

Definición

`setsockopt` establece opciones de un socket. Con **`SO_REUSEADDR`** el socket puede reutilizar la dirección local (IP y puerto) aunque esté en uso o en **TIME_WAIT**.

Se llama **después de `socket()` y antes de `bind()`**, con 5 argumentos: descriptor, `SOL_SOCKET` (nivel socket, no TCP ni AF_INET), `SO_REUSEADDR`, apuntador a un entero con 1 y su tamaño.

```
int activado = 1;
if (setsockopt(fd_s, SOL_SOCKET, SO_REUSEADDR, &activado, sizeof(int)) == -1) {
    perror("Error en setsockopt");
    exit(1);
}
```

Sin la opción, al matar el servidor con Ctrl+C y relanzarlo aparece **`address already in use`**. No permite dos servidores activos en el mismo puerto. Demostración: Programa 05 en el puerto 4898; el chat es síncrono (petición 1, respuesta 1…) y los mensajes de más se acumulan en el búfer.

2. TCP frente a UDP

TCP	UDP
Confiable: flujo de bytes **en orden**	No garantiza llegada ni orden
Fragmenta mensajes largos; si de A B C D E falta C, retiene D y E, pide reenviar C y entrega todo en orden	"Volantes por la ventana"

Error común: suponer que TCP entrega un mensaje largo en un solo bloque.

3. Qué limita una lectura

El kernel guarda lo recibido en un búfer interno; si solo hay 4096 bytes disponibles, `read()` devuelve como mucho 4096 aunque pidas más. Lo que entra depende del emisor, la red y el búfer del kernel.

```
cat /proc/sys/net/core/rmem_default    # en clase: 212992 (~212 KB)
cat /proc/sys/net/core/rmem_max        # en clase: 212992
```

Regla obligatoria ("en esto fallan muchos")

TCP no garantiza recibir todo de una vez: **leer en bucle hasta que `read()` devuelva 0**.

```
while ((n = read(fd, buffer, sizeof(buffer))) > 0) {
    /* procesar o acumular el bloque */
}
```

Tamaño de búfer	Uso
Potencias de 2 (128 … 16384)	Siempre
4 KB (4096)	Recomendación estándar: **tamaño de una página de memoria**
8–16 KB	Tráfico web y transferencias grandes
≥ 32 KB	Redes muy rápidas
Gigantesco (100,000)	No rinde más si el kernel es menor; desperdicia RAM con muchas conexiones

4. compara_tiempos_buffers.c

1. `main()` llama a `medir_tiempo(dominio, tam)` con el dominio de `argv` y búferes de 1024, 4096, 8192, 16384 y 100000.
2. `medir_tiempo()` crea el socket, resuelve con `gethostbyname()` y hace `connect()` al **puerto 80**.
3. Envía el `GET` de HTTP con `send()`/`write()`.
4. `inicio = clock();` → lee todo en `while` hasta `read() == 0` → `fin = clock();`
5. **`tiempo = (double)(fin – inicio) / CLOCKS_PER_SEC;`**, `free()` y cerrar.

Antes lo simuló con `telnet www.google.com 80` (cabecera HTTP + doble Enter).

Búfer	1 KB	4 KB	8 KB	16 KB	100 KB
Tiempo (1.ª corrida)	0.0015 s	0.0013 s	0.0011 s	0.0011 s	0.0011 s

La página pesó ≈ 90,000 bytes (varía por corrida). En la práctica 8–16 KB pueden rendir mejor que 4 KB. Ojo: `clock()` mide **tiempo de CPU** del proceso, no de reloj; la espera de red casi no cuenta. Error común: correr sin pasar el dominio → acceso a un `argv` inexistente → segfault.

Avisos

- Examen parcial: no escrito; un programa para hacer en casa, entrega por correo el lunes (ACS-E01). Estudiar los Programas 04 y 05.

PARTE 2 · Resumen para estudiar

Versión corta de lo que más probablemente se pregunta. Léela primero y vuelve a la Parte 1 si algo no queda claro.

SO_REUSEADDR (examen)

`setsockopt(fd, SOL_SOCKET, SO_REUSEADDR, &activado, sizeof(int))` entre `socket()` y `bind()`; evita `address already in use` por TIME_WAIT. Validar contra −1.

Leer en bucle. TCP fragmenta y el kernel limita (rmem 212992): `while ((n = read(...)) > 0)` hasta 0.

Dato	Para recordar
Búfer recomendado	4 KB = página de memoria; 8–16 KB web
Medir tiempo	(fin − inicio) / CLOCKS_PER_SEC con `clock()` (tiempo de CPU)
TCP vs. UDP	TCP ordena y retransmite; UDP no
Examen	Parecido a los Programas 04 y 05